

Titan: Efficient Polynomial Commitments from IOPs over Groups

Chethan Kamath
IIT Bombay
ckamath@cse.iitb.ac.in

Ravi Prakash
IIT Bombay
24m2131@iitb.ac.in

Samipa Samanta
IIT Bombay
24d0380@iitb.ac.in

Sruthi Sekar
IIT Bombay
sruthi@cse.iitb.ac.in

Nitin Singh
IBM Research, India
nitisin1@in.ibm.com

A Artifact Appendix

A.1 Abstract

This artifact contains the implementation of Titan, a polynomial commitment scheme (PCS), together with benchmarking infrastructure for evaluating Titan against existing PCS constructions. The artifact measures commitment time, evaluation time, verification time, and proof size.

The artifact also implements the MicroSpartan PIOP instantiated with Titan PCS, providing a transparent SNARK construction. The repository contains the source code, benchmarking experiments, scripts for processing benchmark results, and scripts for generating plots.

The Titan benchmarks produce cleaned CSV files containing measurements for different input sizes. The plotting scripts use these CSV files to generate the figures used to analyze the experimental results.

A.2 Description & Requirements

The artifact is organized as a Rust project. The project root contains the README file and the Titan paper (titan.pdf). The main source code is contained in the src directory.

The main components relevant to artifact evaluation are:

- src/benchmarking/: benchmark scripts, cleaned CSV files, and plotting scripts.
- src/benchmarking/titan_plot.py: Python script for generating plots from the benchmark results.
- src/benchmarking/whirpcs.sh: script used to reproduce the Whir baseline benchmarks.
- src/benchmarking/brakedown_hyrax_benchmarking_steps.txt: instructions for reproducing the Brakedown and Hyrax benchmarks.

The Titan experiments are executed in release mode with Rust test and Rayon parallelism restricted to one thread. On Linux, taskset is used to pin the experiment to a single CPU core.

The artifact does not require a specific GPU or other specialized accelerator. The benchmark configuration is designed for single-core CPU execution.

A.2.1 Security, privacy, and ethical concerns. The artifact does not require access to private or sensitive user data. The experiments operate on locally generated benchmark inputs and do not require network access during normal Titan benchmark execution.

A.2.2 How to access. The artifact is available as source code in the project repository.

The repository can be downloaded as a ZIP archive, extracted locally, and accessed from the resulting directory. Alternatively, users may clone the titan-implementation repository using Git.

[INSERT THE PERMANENT ARTIFACT/DOI LINK HERE IF THE ARTIFACT HAS BEEN ARCHIVED, E.G., THROUGH ZENODO.]

A.2.3 Hardware dependencies. None

A.2.4 Software dependencies. The artifact requires:

- Rust and Cargo, installed using the Rust toolchain.
- Python 3 for generating benchmark plots.
- Git for reproducing the Whir baseline benchmarks.
- taskset for the primary single-core benchmark configuration.

A.2.5 Benchmarks. None

A.3 Set Up

The artifact can be obtained by downloading the repository as a ZIP archive and extracting it locally. It is recommended that the artifacts be run on Linux. After extraction, the user should navigate to the root directory of the project:

```
cd path/to/titan-implementation
```

All benchmarking commands described below and in the Experiments section (A.4.2) should be executed from the root directory of the artifact. The detailed set-up and execution instructions can be found in the README file.

A.3.1 Installation. Install Rust and Cargo using rustup and verify the installation:

```
curl --proto '=https' --tlsv1.2 -sSf \
  https://sh.rustup.rs | sh
rustc --version
cargo --version
```

Python 3 is required for generating plots. It can be installed with:

```
sudo apt update
sudo apt install python3
```

A.3.2 Basic test. As a basic functionality test, execute the MicroSpartan PIOP test with Titan PCS:

```
cargo test test_spartan_titan_index \
  --release -- --nocapture
```

A successful execution prints the setup time, indexing time, proving time, verification time, and proof size.

This test provides a basic check that the Titan PCS and its integration with MicroSpartan are correctly compiled and functioning.

A.4 Evaluation Workflow

The evaluation workflow consists of reproducing the Titan PCS benchmark, generating plots from the benchmark output, and, where desired, reproducing the baseline measurements for the other PCS schemes.

A.4.1 Major claims. The following claims should be mapped to the corresponding claims and experimental results in the paper.

- (C1): Titan achieves constant-size commitments, $O(n)$ commitment time, $O(\sqrt{n})$ evaluation time, and $O(\sqrt[4]{n})$ proof size and verification time. This claim is evaluated using experiment (E1).
- (C2): On Pasta curves, Titan achieves 2.5 faster commitment, 10 faster valuation, and 4 faster verification than Dory, with approximately 25 KB proofs for $n = 2^{20}$ verifiable in < 50 ms. This claim is proven by experiments (E2) and (E3).
- (C3): MicroSpartan PIOP when instantiated with Titan PCS reduces proof size and verification complexity from $O(\sqrt{n})$ to $O(\sqrt[4]{n})$, achieving approximately $3\times$ smaller proofs for $n \approx 2^{23}$ while retaining MicroSpartan’s prover efficiency. This claim is evaluated using experiment (E3).

A.4.2 Experiments. The detailed execution instructions can be found in the README file.

- (E1): Titan PCS Scaling Benchmark [5 person-minutes + 5 compute-hours]

Preparation: Navigate to the root directory of the artifact. Ensure that Rust and Cargo are installed.

Execution: Run the Titan scaling experiment:
`taskset -c 0 env RAYON_NUM_THREADS=1 \`
`RUST_TEST_THREADS=1 \`
`cargo test titan_scaling_experiment \`
`--release -- --ignored \`
`--nocapture --test-threads=1 \`
`| grep '^[0-9]' > \`
`src/benchmarking/titan_clean.csv`

Results: The experiment produces `src/benchmarking/titan_clean.csv`. Each row contains the input size, commitment time, evaluation time, verification time, and proof size.

The expected outcome is a successfully generated `titan_clean.csv` file containing the above measurements for the Titan scaling experiment.

- (E2): Baseline PCS Comparison [30 person-minutes + 25 compute-hours]

This experiment reproduces the benchmark results for the baseline schemes (Brakedown, Dory, Hyrax, Kopis, Whir) included in the artifact. To obtain accurate results, it is recommended to run these experiments sequentially.

Preparation and execution: For Brakedown and Hyrax, follow the instructions in `src/benchmarking/brakedown_hyrax_benchmarking_steps.txt`.

For Dory and Kopis, use their corresponding scaling experiments with the same environment configuration as the Titan benchmark. For Dory, for example, the scaling experiment can be run using:

`taskset -c 0 env RAYON_NUM_THREADS=1 \`

`RUST_TEST_THREADS=1 cargo test \`
`dory_BLS_scaling_experiment --release --\`
`--ignored --nocapture --test-threads=1 | grep \`
`'^[0-9]' > src/benchmarking/dory_BLS_clean.csv`

For Whir, clone the Whir repository:

`git clone \`
`https://github.com/WizardOfMenlo/whir.git`
 Copy the provided benchmarking script into the cloned repository:

`cp src/benchmarking/whirpcs.sh whir/`

Navigate to the Whir repository:

`cd whir`

Run the Whir benchmarking script:

`./whirpcs.sh`

Results: The output files containing commitment time, evaluation time, verification time, and proof size are stored in the respective CSV files for each baseline scheme, including `brakedown_clean.csv`, `dory_BLS_clean.csv`, `whir_clean.csv`, etc. These files can be processed and visualized using the provided Python plotting script:

`python3 src/benchmarking/titan_plot.py`

The resulting PNG plots are stored in `src/benchmarking`.

- (E3): MicroSpartan PIOP with Titan PCS [5 person-minutes + 1 compute-hours]

This experiment evaluates the integration of MicroSpartan PIOP with Titan PCS.

Preparation: Ensure that Rust and Cargo are installed.

Execution: Run:

`cargo test test_spartan_titan_index \`
`--release -- --nocapture`

Results: The output reports setup time, indexing time, proving time, verification time, and proof size.

A.5 Notes on Reusability

The artifact can be reused to evaluate Titan and the baseline PCS schemes under different configurations. For Titan PCS, parameters such as the number of variables and number of runs can be modified in `src/protocols/titanpcs.rs`; for Dory in `src/dory.rs`; for Kopis in `src/protocols/kopis_pcs.rs`; for Brakedown and Hyrax in the `poly_commit` repository; and for Whir in `src/benchmarking/whirpcs.sh`. For MicroSpartan with Titan PCS, parameters such as constraint size and number of queries can be modified in `src/protocols/spartan_titan.rs`.

Execution settings such as CPU affinity and the number of threads can also be modified directly in the benchmark commands, using `taskset`, `RAYON_NUM_THREADS`, and `RUST_TEST_THREADS`.